

Crosscoders for Model Diffing an R1 Distill

Executive Summary

Objective

I explored using crosscoders to analyse differences between [Qwen2.5-Math-1.5B](#) (QM) and the R1 Distill [DeepSeek-R1-Distill-Qwen-1.5B](#) (DS). My aim was to get an idea of how effectively crosscoders can characterise model changes during fine tuning. Fine-tuning for more powerful reasoning capabilities seems like an extremely safety relevant use case for this, as it could help understand the emergence of dangerous capabilities such as deception. Model-diffing finetunes could also help us understand behaviours such as the recent [emergent misalignment](#) result (which I'd love to do these experiments on if I had an A100!).

Experiments Overview

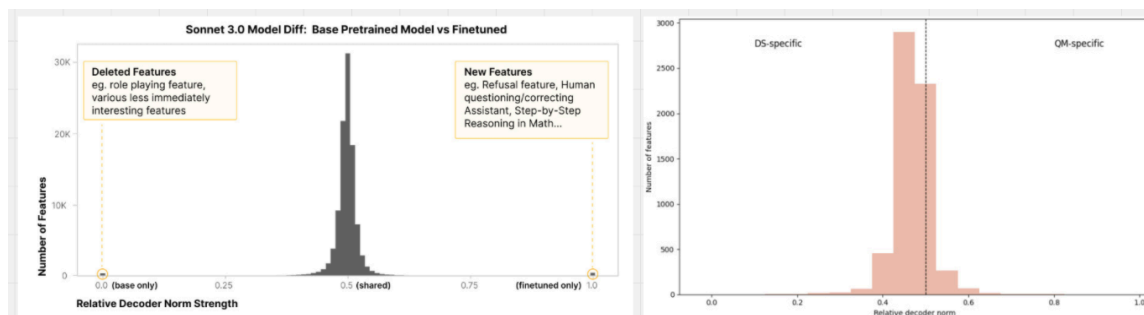
- I build on [this repo](#) and my code is public on GitHub here
- I train crosscoders on the final layers of both models and use an expansion factor of 4 (the largest I can on my GPU). I do some basic hyperparameter tuning to improve the trade of between L0 and mean explained variance (MEV).
- I initially train on the open source [NuminaMath-CoT](#) dataset, but later generate my own because I find what appears to be a bias towards QM features. I do this by collecting QM and DS responses to the Numina Math questions (uploaded here).

measurement.) <think> Okay, so I need to figure out what the Mikhailov family's monthly income should be to save an additional 5% of their current savings while keeping all their other expenses the same. Then, I also need to determine how much they'll save in savings

measurement.) To solve this problem, we need to follow these steps: 1. Calculate the total monthly expenses of the Mikhailov family. 2. Determine the amount of savings needed to save an additional 5% of the monthly income. 3. Calculate the new monthly income required to

Examples from the generated dataset. The DS response (left) is more informal than the QM response, which uses more formal language and structured reasoning steps. These styles are consistent across questions.

- I don't get a trimodal distribution of relative decoder norms (which is how Anthropic extracts their model A specific, shared and model B specific features).



What I wanted (left) vs what I got...

- I brainstorm some reasons why this is the case under [potential issues](#). I expect I would get better results with a larger expansion factor or by enforcing sets of shared and model specific features as Anthropic [recently proposed](#).
- In the absence of a trimodal distribution, I define model specific features as ones which activate when given only the activations of model A, but not when given only the activations of model B (see [Extracting 'model-specific' features](#) section for more detail and some notes on why I think this is flawed but maybe not too awful!)
- I find that increasing sparsity increases the number of these model specific features (according to my activation based definition), but still does not give a trimodal distribution of relative decoder norms.

Active in DS only: 0 Active in QM only: 30 Active in both: 5950 Dead: 164	Active in DS only: 3 Active in QM only: 345 Active in both: 5173 Dead: 623	Active in DS only: 550 Active in QM only: 1257 Active in both: 3947 Dead: 390
--	---	--

Left to Right - tanh sparsity scaling factors 0.5, 1 and 2

- Despite this, Claude does consistently find a trend of more 'informal reasoning' features in the DS specific features I identify when comparing them with the shared or QM features. This aligns with the behaviour I observed when generating the QM and DS dataset responses.

Model A appears optimized for capturing the cognitive processes of mathematical reasoning. It focuses on the natural language expressions that reflect human-like thinking when solving math problems - the metacognitive narration of problem-solving steps, recognition of patterns, retrieval of mathematical knowledge, and explanation of concepts.

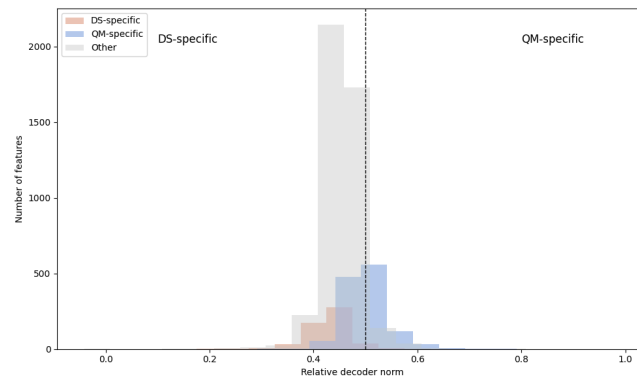
Model B appears optimized for capturing the structural and notational elements of mathematical writing. It focuses on the formal representation of mathematical content - the LaTeX notation, document organization, equation formatting, and symbolic expressions.

These two aspects are complementary and represent different dimensions of mathematical communication:

- Model A captures "how mathematicians think"
- Model B captures "how mathematics is written"

Sonnet 3.7 comparison of features I define as DS and QM specific based on the activations method. See [Exploring Features](#) for similar results from a second chat.

- I also compare my activation-based DS specific features with the 5% of features with the highest relative DS to QM decoder norms. These sets do not align well, as shown by their distributions in the graph below, which I find quite unexpected.
- However, both Claude and GPT4o find more informal reasoning features in the activation-extracted features than the norm-extracted ones. Asking an LLM to compare lists of highest activating feature tokens is not a very reliable means of evaluation! But it may imply that the activation extraction method is somewhat working.



The relative decoder norm distributions of the activation-extracted model-specific features compared to the shared and dead features. The model-specific distributions have slightly higher than average decoder norms for the model they activate for relative to the model they do not activate for, but the difference to be greater.

Feature	Activation	Norm	Overlap
Cognitive Style	Informal, reasoning-oriented	Formal, equation-driven	Mixed (both styles present)
Use of Math Symbols	Some, embedded in thought process	Frequent, direct usage	Mixed
Common Phrases	"Let me think," "Wait," "Hmm"	"To solve this problem," "We need to determine"	Both styles appear
Focus	Step-by-step problem-solving	Direct equation solving	Hybrid of explanation and equation-solving

A nice table GPT 4o made comparing the features in each set, see [Comparing model-specific feature extraction methods](#) for a similar response from Claude.

Some Takeaways

I am both pleasantly surprised that I managed to extract some seemingly model specific features, and disappointed that these are not cleanly distinguishable by their decoder norms. Having done this work, I do think crosscoders seem promising for model diffing, but my training would have to be greatly improved to make these useful. A larger expansion factor and designating shared features as mentioned above could help a lot with this. There are also a few other unexplained trends I found training these crosscoders - for example when I increase sparsity the MEV decreases much faster on the DS model than the QM one, which it would be helpful to try and understand.

Having looked at some of the crosscoder features, I increasingly think that examining features themselves may not be sufficient to get a good idea for model changes. Using them to initially locate feature changes, as a starting point for identifying circuits or quantifying feature interactions, may be necessary to understand more advanced behavioural changes.

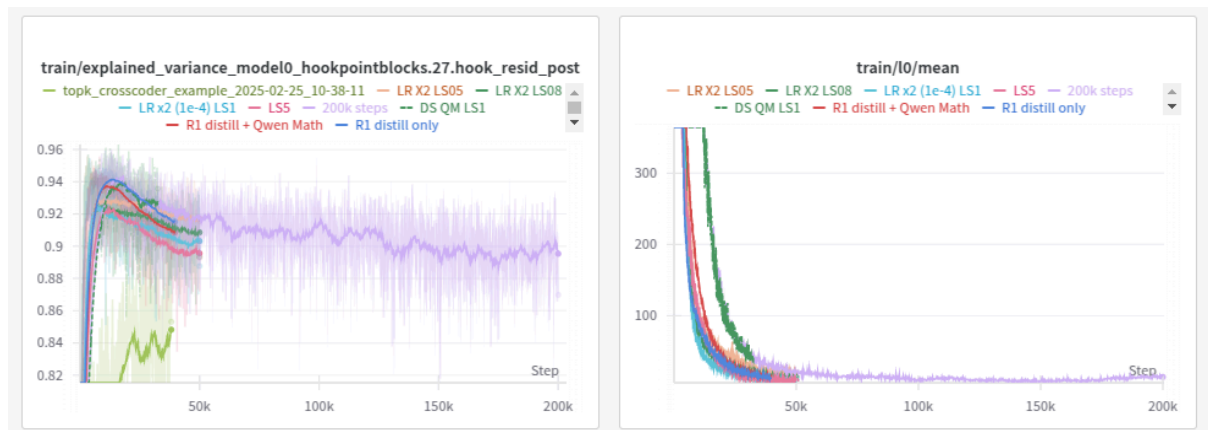
I got the impression that changing the dataset did quite significantly alter the crosscoder (which makes sense) so it would be interesting to directly compare these results to a stagewise model diffing approach.

V1 - Numina CoT Data

I'm using the smallest R1 distill because of compute restrictions. For the same reason I train on only one layer in each model (blocks.27.hook_resid_post) with an expansion factor of 4 (giving 6144 latent features). The layer is a somewhat arbitrary choice, but is motivated by the idea that features may become more 'specialised' in later layers¹, which may give to clearer differences between the models.

I don't have access to the fine-tune dataset or any information about it other than that it was generated from the full scale R1. I also have no access and limited info about the Qwen2.5-Math dataset. In the absence of these, [NuminaMath-CoT](#) dataset seems somewhat appropriate for the crosscoder training because it is large and diverse enough to avoid overfitting and it uses chain of thought in the responses.

The crosscoders are JumpReLU with a tanh sparsity loss and they use the data dependent initialisation method recommended [here](#).

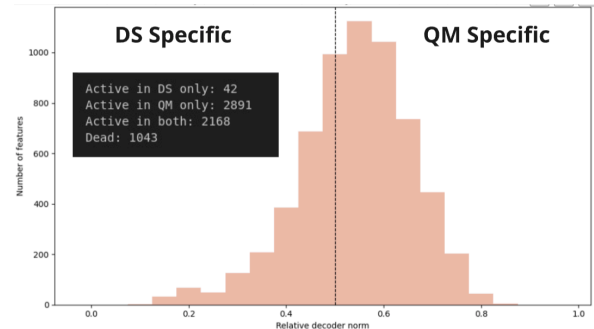
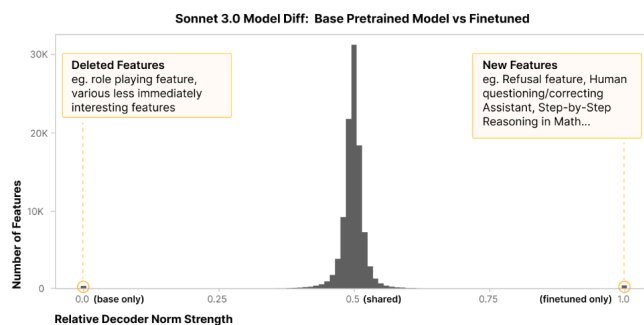


I did a very small hyperparameter search to get an appropriate tanh scaling factor (λ) and learning rate to get a good balance of L0 and mean explained variance², and to see how few training steps I can get away with. I'm unsure what L0 I should actually be aiming for, since with the small expansion factor I am using I'm unsure whether greater sparsity will increase or decrease how monosemantic features are. I initially look at the crosscoder trained with $\lambda = 0.8$ and LR $1e-4$, which gives L0 ~ 15 and MEV ~ 0.9 (green lines on the training curves).

Unfortunately, looking at the distribution of relative decoder norms, I don't get the nice trimodal distribution found in the Sonnet finetune model-diffing results in the original crosscoder post. (I also don't get it when plotting with a log scale y axis). I also find my distribution is skewed towards QM features.

¹ Evolution of SAE Features Across Layers in LLMs - <https://arxiv.org/html/2410.08869v1>

² MEV here is implemented as variance per residual stream dimension rather than using L2 norms across the vector, which I think probably isn't great as it means dimensions contribute equally regardless of magnitude. I didn't realise this till the end, but since I don't rely on the MEV measure for any results I don't expect this would have changed anything I did.



Extracting 'model-specific' features

Without a separation in decoder norms, defining 'QM only' and 'DS only' features seems hard. I could use a top and bottom X% of the features by relative decoder norms, but this is somewhat arbitrary. Instead I try to classify features based on activation patterns:

- I get model activations from 500 prompts (512*500 tokens) and pass these through the crosscoder in 3 ways: with activations from both models, and with activations from only one model, with the other model activation vector set to 0.
- I define dead features as those that never activate and shared features as those which activate in both one-model settings.
- I classify features as model-specific if they activate in only one of the one-model settings.

This seems like a flawed way of detecting these features because having a model's activation vector set to 0s is clearly unnatural, but I am unsure how to do it better! I do find that these categories do add up to the total number of features, which means that all features that activate on the full activation vectors from both models also activate in both of the model 1 activations only and model 2 activations only settings. I considered setting the second activation vector to its mean instead, but since I expect features specific to model A to have below average activation magnitudes in model B, I don't think this is any more valid.

Looking at the shared features, there are both seemingly monosemantic ones (e.g. left) and seemingly polysemantic ones (e.g. right). These lists are generated by collecting the top 5 activating tokens for each feature across 500 prompts. **I format top activating tokens in square brackets (ie. [token]) and show them within their local context.**

```
Feature 12269
DS:
+ \cos^4 \alpha$ is $\[q]quad$
bf{(C)}\ 125\[q]quad\textbf{(D)}
bf{(B)}\ 100\[q]quad\textbf{(C)}
\textbf{(A)}\ 50\[q]quad\textbf{(B)}
QM:
bf{(C)}\ 125\[q]quad\textbf{(D)}
bf{(B)}\ 100\[q]quad\textbf{(C)}
+ \cos^4 \alpha$ is $\[q]quad$
\textbf{(A)}\ 50\[q]quad\textbf{(B)}
```

```
Feature 12248
DS:
the second bakery needs 4 sacks, and the[ third] bakery needs 12 sacks of
-elimination tournament. (A single-[elim]ination tournament is one where once a
12 bottles in 1 minute. Machine B[ can] put caps to 2 fewer bottles
QM:
. The first bakery needs 2 sacks, the[ second] bakery needs 4 sacks, and
the second bakery needs 4 sacks, and the[ third] bakery needs 12 sacks of
smaller than the arithmetic mean of numbers $a[, ] b, c, d$ ?
, they spliced Petya's video into[ V]asya's. The result was
James has 20 pairs of red socks and[ half] as many black socks. He
```

The model specific features largely seem to be polysemantic, e.g. :

```
Feature 11907
DS:
defined on  $\mathbb{R}$ , [ with] real numbers  $x_1$ 
function defined on  $\mathbb{R}$ , [ with] real numbers  $x_1$ 
 $(x-a) = x^3 + 1$ , [ and] for any real number  $x$ ,  $f$ 
QM:
```

```
Feature 12266
DS:
QM:
 $a_{n+2} = 4a_{n+1} + a$ 
In the post-apocalyptic wasteland, 1 lizard is
same point on the path in both videos, they spliced Petya's video into
 $\frac{x}{3} = \frac{y}{5} = \frac{z}{7}$ 
be the sequence of integers defined recursively by  $a_0 = 0$ ,  $a$ 
```

And when they are monosemantic they don't seem like they should be model specific!

```
Feature 518
DS:
In[ the] Cartesian coordinate system, the distance from
In[ the] sequence  $a_n$ 
In[ the] geometric sequence  $a_n$ 
In[ the] polar coordinate system, the equation of
In[ the] post-apocalyptic wasteland,
QM:
```

```
Feature 5
DS:
QM:
Given[ that]  $y = f(x)$ 
Given[ that] the function  $f(x)$  satisfies  $f$ 
Given[ that] the variance of the data  $a$ 
Given[ that] the sequence  $a_n$ 
Given[ that] the sequence  $a_n$ 
```

I don't do any more principled feature analysis here, because the features I inspect manually seem uninteresting for model diffing and because the distribution of the relative decoder norms suggests that I need to improve my crosscoder training!

V2 - Custom Dataset

Generating the Dataset

There appears to be a bias towards QM features: the relative decoder norms are skewed towards QM and I have nearly 70x more QM specific features than DS specific ones. I think this may be because the DS specific behaviour isn't well reflected in the dataset, so I'll try and fix this.

To get a more appropriate dataset, I generate a set of responses from the QM and DS models. I load the problems from the NuminaMath dataset, locally generate model responses from the QM and DS models and save these to a new dataset. Based on the R1 distill usage advice, I add `<think>` to the end of each problem when generating DS responses. I save responses to a csv file and let it run overnight, which gives me 13k sequences. This dataset is on Hugging Face here. It's useful to see the distinct style difference in responses: both models break problems down into clear steps, but DS (left text examples) responds in a more conversational and less certain style than QM (right).

```
triangle, then  $(R < r)$ . <think> Okay, so I have
this problem here about triangle ABC and its
orthocenter H. They mention the circumradius of
triangle ABC is R, and the circumradius of triangle BHC
is r. I need to figure out the relationship between R
```

```
measurement.) <think> Okay, so I need to figure out
what the Mikhailov family's monthly income should be to
save an additional 5% of their current savings while
keeping all their other expenses the same. Then, I also
need to determine how much they'll save in savings
```

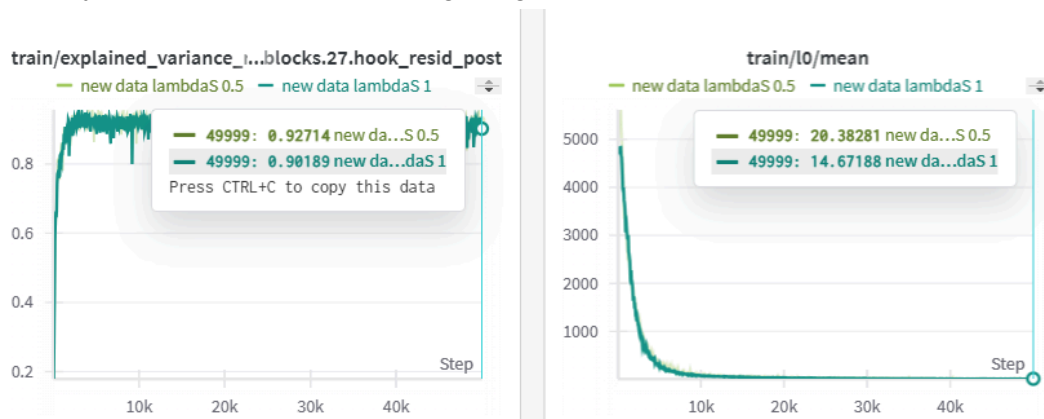
```
triangle, then  $(R < r)$ . (E) None of the above.
(27th American High School Mathematics Examination,
1976) To solve this problem, we need to understand the
geometric properties of the orthocenter and the
circumradius of a triangle. Let's break down the
```

```
measurement.) To solve this problem, we need to follow
these steps: 1. Calculate the total monthly expenses of
the Mikhailov family. 2. Determine the amount of
savings needed to save an additional 5% of the monthly
income. 3. Calculate the new monthly income required to
```

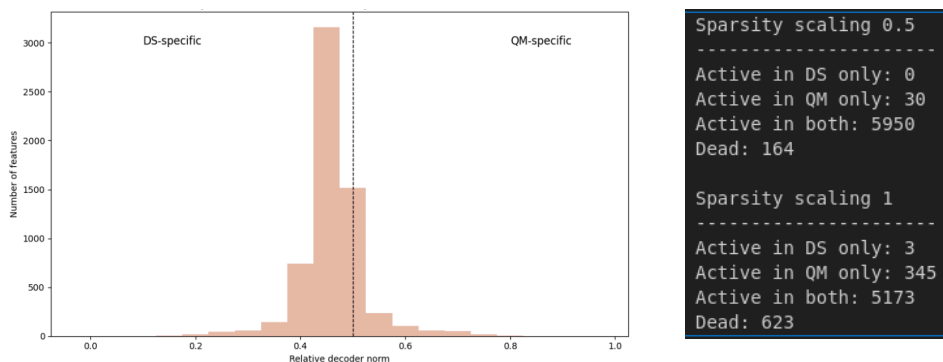
In doing this I also found that using the Qwen tokenizer (which I've been using for crosscoder training so far) can cause the DS model to respond strangely. For example it will sometimes just output the `</think>` token repeatedly. Because of this I'm switching to using the modified DS tokenizer for the crosscoder data loading. The tokenizers seem to only be minorly different (based on a quick look at the json files) but the DS version does add various special tokens including the `<think>` and `</think>` ones.

New Crosscoders

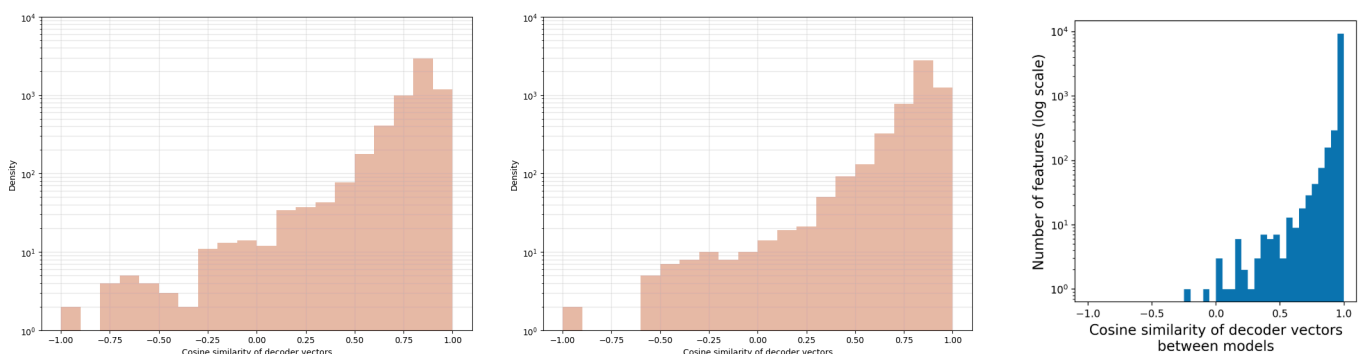
I trained two versions with the new dataset and changed tokenizer, comparing different sparsity levels ($\lambda=2$ and $\lambda=4$). I'm getting a similar MEV and mean L0 to before.



The distribution of relative decoder norms has narrowed and shifted towards the distilled model (see below), which appears to support the idea that there were insufficient DS relevant features in the previous dataset and the new dataset has helped this. However, I am still not seeing a trimodal distribution and am getting far fewer model-specific features than before, with no apparent improvement in the ratio of DS specific to QM specific ones.



This relative decoder norm distribution is interestingly near identical with the different sparsity levels. When I also look at the cosine similarity of the decoder vectors between models, I am also getting a much 'messier' result than the Anthropic team found in their Sonnet model diff. These graphs (L to R) show my CCs with sparsities 0.5 and 1 and the Anthropic result, which unlike mine has practically no <0 results.



Potential Issues

The [Anthropic Crosscoder update](#) finds that single model crosscoder features are frequently more polysemantic than shared features because they need to encode more information in order to 'justify' their presence. They find they can get more monosemantic model-specific features by allocating designated 'shared' and 'exclusive features' (by enforcing decoder weight or norm sharing and decreasing sparsity penalty in the shared features).

Based on this and some other ideas I see a few possible options for why I am struggling to get clean model-specific features:

1. The issue identified in the above post where shared features outcompeting single model features. I could try and implement Anthropic's fix but expect I'd run out of time.
2. I don't have enough features. The small crosscoder dimension probably makes all of the above issues worse, but this isn't something I have to RAM to fix. Anthropic train a 1 million feature crosscoder for their Sonnet results, which could be around an 80x expansion factor. I can't do this sadly :(
3. As discussed in the previous section, I am probably not detecting model specific features very well.
4. The models could actually be very different, which may make it challenging to learn anything meaningful without having sufficient features to capture both models. I don't know how DS was finetuned and for how long.
5. Similar to 2, but the fact that I am diffing the final layer may mean I am trying to compare more specialised features, of which there may be more.
6. There could be something very flawed in my implementation! This is all rapidly hacked together and I may just have made an error somewhere which is causing the crosscoders to perform poorly. Easiest way to check this would probably be to apply the same method to a very toy example of a base and fine-tuned model but this is probably also out of scope in the time I have

Several of these fixes are out of scope for me here but I think increasing sparsity might help slightly as it may give me more monosemantic and/or model specific features. There seems to be a slight trend towards this in the 2 crosscoders above, where the sparser crosscoder has more model-specific features. I should also look more closely at how I extract features and compare it with the decoder norm approach.

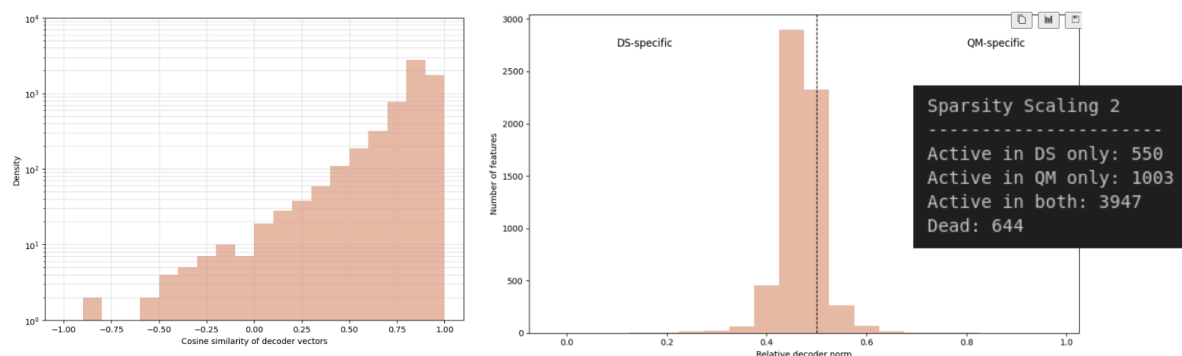
Increasing Sparsity

I train crosscoders with $\lambda = 2$ and $\lambda = 4$. For $\lambda = 2$, final mean L0 goes down to 6.2 while for $\lambda = 4$ it goes down to 5.2. Strangely the MEV decreases significantly ($\sim 4x$) more for the DS model than the QM model.



Seemingly the crosscoder is ‘choosing’ which model to represent well given its restricted capacity. This might be fixable with an improved method for scaling losses between the different models, or by defining specialised features for each model similar to how Anthropic recently proposed (as mentioned above).

The increased sparsity does not significantly change the distribution of cosine similarities or relative decoder norms, but does increase the number of single model features, however, giving 550 and 1003 for DS and QM respectively with $\lambda = 2$. I decided to analyse $\lambda = 2$ rather than $\lambda = 4$ in depth since the difference in L0 seems relatively minor between the two.



Exploring features

Since this is now too many to spot trends manually, I ask Claude to identify any trends in the type of features present in the DS specific set. It notes several categories of math/syntax related tokens as well as the presence of reasoning related features, but not in a way that seems specific to the DS model.

Reasoning Phrases

- **Feature 892:** Activates on phrases that introduce analytical steps (‘[.

]Let me write down the number:’)

- **Feature 2477:** Recognizes phrases that indicate understanding of problems (‘[.

]”A math club consists of 8’)

- **Feature 151:** Detects reasoning transitions (‘[.

]Let’s break down the problem into’)

Feature 892

DS:

|divisible by some smaller primes and work my way up[.

]Let me write down the number:

|that $\sin(\theta)$ equals $\sqrt{2}/2$ [.

]Now, I remember that $\sqrt{x^2 + y^2}$

|& $x+y$ \end{vmatrix}.

\[

]Hmm, determinants can sometimes be

{ $a^2 + b^2$ }\).

]Let me make sure I’m recalling

Feature 2477

DS:

|read the problem again to make sure I understand it[.

]”39 persons can repair a

|me read it again to make sure I understand it[.

]”A math club consists of 8

|to understand it step by step. The problem says[:

]”As shown in Figure 1,

|me try to break it down step by step.[

]First, the problem says: "

|Let me try to understand it step by step.[

From looking through some of the features manually, there are some features which seem to correspond to DS specific behaviours better, such as a feature for the first person ‘I’ and one which activates on punctuation before ‘Hmm’.

Feature 231

DS:

5 meters, what is the height of the building[?] Hmm, okay,
What is the percentage of the stock he invested in[?] Hmm,
probability that $x + y$ is less than 4[.] Hmm, probability
that $3/2$ of all chocolates are eaten[.] Hmm, that sounds a
represents a circle with a radius of 10[.] Hmm, I remember

Feature 1052

DS:

|me think about how to approach this.

First,[I] remember that when dealing with large ex
's think about how to approach this.

First,[I] remember that when expanding a polynomial raised
'll start by approximating $\sqrt{20}$.

[I] know that $\sqrt{16}$ is

than 2. Hmm, let's see.[I] remember that the GCD of a
me break this down step by step. First,[I] remember that a

However, when I put in the QM specific features, I also get some tokens identified as related to problem solving. These seem to be mono-semantic and activate on specific words such as ‘step’ and the ‘s’ in ‘let’s’ (feature 5186). I do also find a QM specific ‘I’ feature.

Problem-Solving Approaches

- **Feature 5406:** Detects step-by-step problem breakdowns
- **Feature 5897:** Recognizes reasoning methodology descriptions
- **Feature 5868:** Identifies computational approaches, particularly Python-based solutions
- **Feature 5636:** Activates on variable solving steps in geometry problems
- **Feature 5186:** Focuses on expressions starting with "Let's" that introduce solution approaches

```
Feature 5406
DS:
QM:
and the conditions provided. Let's break it down[ step]-by-step:

1. **Under
To solve this problem, let's break it down[ step]-by-step:

1. **Define
some basic algebra. Let's break down the problem[ step]-by-step:

1. **Define
of Miracles. Let's break down the problem[ step]-by-step:

1. Let's
the unknowns. Let's break down the problem[ step]-by-step:
```

```
Feature 600
DS:
QM:
| symmetry but is mapped onto itself by some rotation?
| [I] think the answer is no, but
| of children and adults at today's matinee.

| [I] know there are 48 children
| but I'm not sure how to prove it. [I] know that if a figure has an
| {6} \times (-36)\).

| [I] notice that the expression inside the parentheses
| than 2. Hmm, let's see. [I] remember that the GCD of a
```

This isn't very reassuring, but when I explicitly ask Claude to identify reasoning related features and compare them across the two sets I do actually get a promising answer!

Comparing the reasoning-related features between the two datasets, I do notice some differences in their distribution and emphasis:

Differences in Reasoning Features Between Sets

Dataset 1 (first document)

- More frequent use of the `<think>` tag to mark explicit reasoning processes
- More prevalent "first-person" reasoning patterns ("I need to figure out," "I remember that")
- Greater emphasis on initial problem framing and setup
- More instances of uncertainty markers ("Hmm," "let me think about")
- More exploratory reasoning patterns with multiple approaches considered

Dataset 2 (second document)

- More structured step-by-step reasoning indicated by numbered steps (e.g., "1.", "2.")
- More formal mathematical reasoning approaches
- More instances of technique-specific reasoning (e.g., "Use the Law of Sines")
- More direct algorithmic problem-solving rather than exploratory thinking
- Greater emphasis on implementation details, especially in programming contexts

Other Notable Differences

- Dataset 1 appears to contain more problem setup and initial reasoning phases
- Dataset 2 seems to have more execution and calculation phases of problem-solving
- The reasoning in dataset 1 feels more "human-like" with hesitations and revisions
- Dataset 2 contains more direct application of known algorithms and formulas
- In dataset 1, reasoning tends to be more verbalized and explicit

These differences suggest that the datasets might represent different phases of reasoning or different contexts for problem-solving. Dataset 1 might capture more of the initial problem understanding and approach formulation, while dataset 2 might focus more on the execution of solution strategies and formal reasoning steps.

In a different chat window:

Model A appears optimized for capturing the cognitive processes of mathematical reasoning. It focuses on the natural language expressions that reflect human-like thinking when solving math problems - the metacognitive narration of problem-solving steps, recognition of patterns, retrieval of mathematical knowledge, and explanation of concepts.

Model B appears optimized for capturing the structural and notational elements of mathematical writing. It focuses on the formal representation of mathematical content - the LaTeX notation, document organization, equation formatting, and symbolic expressions.

These two aspects are complementary and represent different dimensions of mathematical communication:

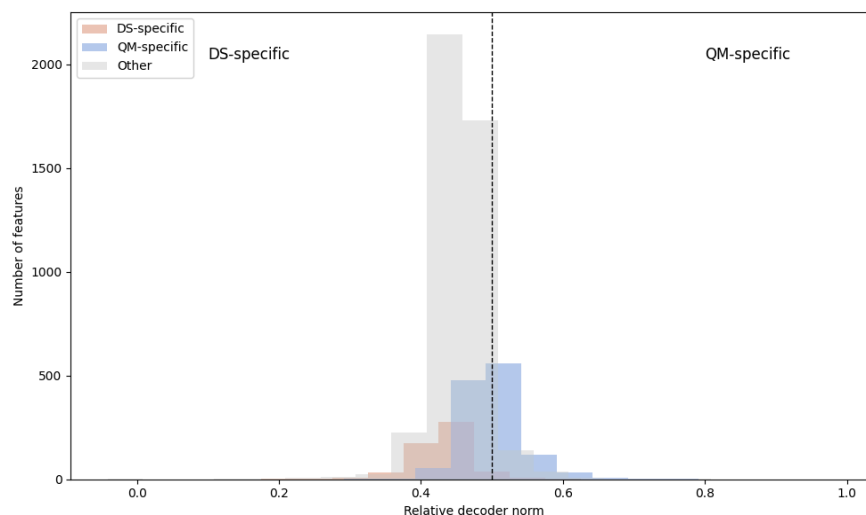
- Model A captures "how mathematicians think"
- Model B captures "how mathematics is written"

This distinction suggests the models might have been trained on different aspects of mathematical content, or that they were fine-tuned to focus on different elements of mathematical problem-solving - one emphasizing the reasoning process and one emphasizing the formal representation.

This analysis could be valuable for understanding how large language models process and generate mathematical content, and how different training approaches might emphasize different aspects of mathematical communication.

Comparing model-specific feature extraction methods

To compare my activation based approach to classifying features with a relative decoder norm thresholding approach, I separately plot the distributions for each of the DS, QM, shared and dead feature sets. The model-specific feature sets do have slightly shifted distributions compared to the shared and dead features, but only slightly, and they definitely don't align with what you would get extracting features based on model percentiles.



To see which approach is 'better' I ask Claude to compare the features across 3 sets:

1. DS specific features extracted based on activations in the way I have done so far;
2. The 5% of features with the highest relative DS to QM decoder norms; and
3. the overlap between these sets.

Interestingly I find that the activation extracted version seems more specific to the DS informal reasoning style.

Frequency Analysis:

Set 1 (Activation Extracted):

- Highest concentration of reasoning language
- Frequent use of "Let me try to understand..."
- Many instances of "break it down step by step"
- Numerous "Hmm..." reflective thinking markers
- Estimated Frequency: Very High (40-50% of features)

Set 2 (Norm Extracted):

- Moderate concentration of reasoning language
- More structured reasoning phrases
- Fewer conversational reasoning markers
- Fewer "Hmm..." reflective moments
- Estimated Frequency: Moderate (25-35% of features)

Set 3 (Overlap):

- Lowest concentration of reasoning language
- More technical, less narrative reasoning
- Minimal conversational reasoning markers
- More direct problem-solving language
- Estimated Frequency: Low (15-25% of features)

GPT 4o agrees and makes a handy table!

Feature	Activation	Norm	Overlap
Cognitive Style	Informal, reasoning-oriented	Formal, equation-driven	Mixed (both styles present)
Use of Math Symbols	Some, embedded in thought process	Frequent, direct usage	Mixed
Common Phrases	"Let me think," "Wait," "Hmm"	"To solve this problem," "We need to determine"	Both styles appear
Focus	Step-by-step problem-solving	Direct equation solving	Hybrid of explanation and equation-solving

Different layers

Since this is very quick to check, I train a couple of CCs on different layers. I arbitrarily select layers 3 (CC3) and 13 (CC13) to have something near the start and middle of the model and because I like the numbers. I train both with $\lambda = 2$.

CC3 reaches a mean L0 of 6.6 and MEVs of 88.1 (DS model) and 90.0 (QM model).
CC13 reaches a mean L0 of 7.6 and MEVs of 80.8 (DS model) and 82.8 (QM model).
(For immediate reference CC27 was mean L0 of 6.2 with MEVs of 0.838 and 0.884).

Still no nice trimodal distribution. However I do find it interesting that the discrepancy between reconstruction on the different models is lower in these earlier layers. My best hypothesis for this is that the models are more similar in earlier layers, which may also be supported by the fact that I get fewer model-specific features in earlier layers, and slightly higher cosine similarities between features.

